UNIT-II

Server Side Programming: servlet – strengths – Architecture – Life cycle – Generic and HTTP servlet – Passing Parameters – Server Side Include – Cookies – Filters.

JSP – Engines – Syntax – Components – Scriplets – JSP Objects – Actions – session trackings – Session Tracking – Database connectivity. Sql statements – J2EE – Introduction – Beans – EJB.

2 MARKS

1. What is a Servlet?

Java Servlets are server side components that provides a powerful mechanism for developing server side of web application. Earlier CGI was developed to provide server side capabilities to the web applications. In CGI played a major role in the explosion of the Internet, its performance, scalability and reusability issues make it less than optimal solutions.

2. What are the types of Servlet?

There are two types of servlets,

i) GenericServlet
ii) HttpServlet

- **GenericServlet**defines the generic or protocol independent servlet.
- **HttpServlet** is subclass of GenericServlet and provides some http specific functionality link doGet and doPost methods.

3. How can we refresh servlet on client and server side automatically?

On client side we can use Meta http refresh and server side we can use server push.

4. What is the difference between ServletContext and ServletConfig?

- ✓ **ServletContext:** Defines a set of methods that a servlet uses to communicate with its servlet container. The ServletContext object is contained within the ServletConfig object, which the web server provides the servlet when the servlet is initialized.
- ✓ **ServletConfig:** The object created after a servlet is instantiated and its default constructor is read. It is created to pass.

5. What are Applets?

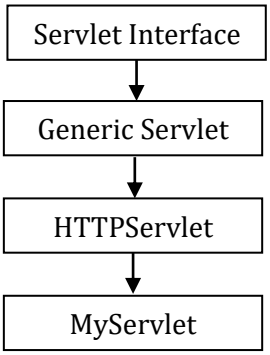
A program designed to be executed from within another application. Unlike an application applets cannot be executed directly from the operating system. With the growing popularity of OLE, applets are becoming more prevalent. A well-designed applet can be invoked from many different applications

6. Differentiate between Servlet and Applet?

- ✓ Servlets are server side components that executes on the server whereas applets are client side components and executes on the web browser.
- ✓ Applets have GUI interface but there is not GUI interface in case of Servlets.

7. Define the Architecture of Servlet?

A Servlet is a class, which implements the javax.servlet.Servlet interface. However instead of directly implementing the javax.servlet.Servlet interface we extend a class that has implemented the interface like javax.servlet.GenericServletor javax.servlet.http.HttpServlet.



8. List the life cycle of servlet?

The javax.servlet.Servlet interface defines the life cycle methods of servlet such as init(), service() and destroy(). The Web container invokes the init(), service() and destroy() methods of a servlet during its life cycle. The sequence in which the Web container calls the life cycle methods of a servlet.

9. What do you mean by server side?

Occurring on the server side of a client-server system. In the World Wide Web, CGI scripts are server-side applications because they run on the web server. In contrast, JavaScript scripts are client side because they are executed by your browser. Java applets can be either Server-side or Client-Side depending on which computer executes them.

10. What is meant by Generic Servlet?

GenericServlet class defines a protocol-independent (HTTP-less) servlet. However while building a website or an online application, you may want to have Http protocol, in that case extend HttpServlet instead of GenericServlet class.

11. What is meant by HTTP Servlet?

The HttpServlet class implements Serializable interface and extends GenericServlet class. It provides the definition of HTTP specific methods such as doGet(), doPost(), doTrace() etc.

12. Compare Servlet with CGI?

Servlets have several advantages over CGI:

- A Servlet does not run in a separate process. This removes the overhead of creating a new process for each request.
- A Servlet stays in memory between requests. A CGI program (and probably also an extensive runtime system or interpreter) needs to be loaded and started for each CGI request.
- There is only a single instance which answers all requests concurrently. This saves memory and allows a Servlet to easily manage persistent data.

13. Define Passing parameter?

The number of parameters may be passed to the servlet using the <PARAM> tag. The servlet can retrieve the parameter values using the getParameter() method of ServletRequest. Any number of initialization (init) parameters may also be passed to the servlet appended to the end of the <SERVLET> tag.

14. What is Server Side Include?

Server Side Includes (**SSI**) is a simple interpreted server-side scripting language used almost exclusively for the Web. The most frequent use of SSI is to include the contents of one or more files into a web page on a web server.

15. What is meant by cookies?

A cookie is a small amount of information sent by the web server which is stored on a user system for lateral retrieval. The major use of cookies is to store user identification details such as name and password.

16. What are the issues in cookies?

The issues in cookies are:

- ✓ Inform users for cookie usage.
- ✓ Avoid using too many cookies
- ✓ Provide alternate for cookie denying users.

17. What is the use of cookies?

- Cookies are short pieces of data sent by web servers to the client browser.
- The cookies are saved to clients' hard disk in the form of small text file.
- Cookies help the web servers to identify web users, by this way server tracks the user.
- Cookies pay very important role in the session tracking.

18. List the various cookie class?

- cookie has
- a name,
- a single value, and
- optional attributes such as a
  - ✓ comment,
  - ✓ path and
  - ✓ domain qualifiers,
  - ✓ a maximum age, and
  - ✓ a version number

19. Define Filters?

A filter is an object that is invoked at the preprocessing and post processing of a request. It is mainly used to perform filtering tasks such as conversion, logging, compression, encryption and decryption, input validation etc. The servlet filter is pluggable, i.e. its entry is defined in the web.xml file, if we remove the entry of filter from the web.xml file, filter will be removed automatically and we don't need to change the servlet. So maintenance cost will be less.

20. Define JSP?

Java Server Pages (**JSP**) is a Java technology that allows software developers to dynamically generate HTML, XML or other types of documents in response to a Web client request. The technology allows Java code and certain pre-defined actions to be embedded into static content.

- ✓ Java Server Pages (JSP) is an extension of servlet technology.
- ✓ JSPs simplify the delivery of dynamic web content.
- ✓ They allow web programmers to create dynamic content by reusing predefined components and by interacting with components using server-side scripting.

21. How JSP works?

1. Translates the JSP source code into the servlet source code.
2. It complies the servlet source code to generate a class file.
3. Loads the class files and create an instance.
4. Initializes the servlet instance by calling the jspInit() method.
5. Invokes the jspService() method, passing the request and response objects.

22. List the life cycle methods of a Servlet?

The javax.servlet.Servlet interface defines the three methods known as life-cycle method.

1. init() method
2. service() method
3. destroy() method

23. What are the components of JSP?

JSP page is built using components such as

- ✓ Directives
- ✓ Scripting Elements
- ✓ Expressions
- ✓ Declarations
- ✓ Scriptlets
- ✓ Standard Actions
- ✓ Custom Tags

24. Write down the various attributes for the page directives in JSP?

The page directive defines information that will be globally available for that Java Server Page,

1. language
2. extends
3. import
4. session
5. buffer
6. content type

25. What are the Life cycles of JSP?

- ✓ JspInit()
- ✓ JspService()
- ✓ JspDestroy()

26. What is the use of Scriptlets?

- We can embed any amount of java code in the JSP Scriptlets.
- JSP Engine places these codes in JspService () methods.

Variables available to the JSP Scriptlets are

- ✓ Request – HttpServletRequest
- ✓ Response – HttpServletResponse
- ✓ Session – HTTP session object associated with the request
- ✓ Out - is an object of output stream and is used to send any output to the client.

27. Define JSP engines?

- To Process JSP page, a JSP engine is needed.
- JSP Engine is nothing but a specialized servlet, which runs under the supervision of the servlet engine.
- The JSP Engine is typically connected with a web server or can be integrated inside a web server or an application server.

  Many servers are:

  1. Tomcat
  2. Java Web Server
  3. WebLogic
  4. WebSphere

28. Define JSP syntax?

The syntax of JSP is almost similar to that of XML. The general rules are

a) Tags must have their matching end tags.
b) Attributes must appear in the start tags.
c) Attribute values in the tag must be quoted.

29. What are the types of directives?

- A Directives element in a JSP page provides global information amount a particular JSP page.

The syntax of jsp directive is

<%@ directive-name attribute="value" %>

The three standard directives available in all JSP environments:

1. page
2. include
3. taglib

30. Write the syntax of page directives?

- The page directive defines attributes that notify the web container about the general settings of a JSP page.

It has the following syntax:

<%@ page attribute="value" attribute="value" %>

Example:

<%@ page language="java" %>

<%@ page language="java" import="java.sql.*" %>

31. Write the syntax of include directives?

- Include directives are too used to specify the name of the files to be inserted during the compilation of the JSP page.
- This directive is to include the HTML, JSP or Servlet file into a JSP file.
- This is a static inclusion to a file.

The Syntax is

<%@ include file="filename" %>

Example:

<%@ include file="/header.html" %>

<%@ include file="/doc/legal/disclaimer.html" %>

<%@ include file="sortmethod" %>

32. What are the types of scripting elements?

The 3 different types of Scripting Elements are

1. Expressions
2. Declarations
3. Scriptlets

33. Write the syntax of Expressions?

The Syntax of JSP Expression is

<% = expression %>

- ✓ JSP Expression start with <% = and ends with %>.
- ✓ Between these, any number of expressions can be embedded.
- ✓ The result of expression is converted to the string to get displayed.

Example:

<%="Hello World!" %>

34. Write the syntax of declarations?

The syntax of JSP Declaratives is

<%!

// declare all the variables here

%>

- ✓ JSP declaration begins with <%! and ends with %>.
- ✓ We can embed any amount of java code in the JSP declarations.
- ✓ Variables and functions in the declarations are class level and can be used anywhere in the JSP page.

35. What is meant by session tracking?

A session is basically a conversation between a browser and a server. All the above technologies can save information for the current session for a particular user visiting a site. The session is important, as HTTP is a stateless protocol. This means that the connection between web server and a web browser is not automatically maintained, and that the state of a web session is not saved.

36. Explain the directory structure of a web application?

The directory structure of a web application consists of two parts.

- A private directory called WEB-INF.
- A public resource directory which contains public resource folder.

WEB-INF folder consists of

1. web.xml 2. Classes directory 3. Lib directory

37. What are the common mechanisms used for session tracking?

- ✓ Cookies
- ✓ SSL sessions
- ✓ URL- rewriting

38. How the Sessions can be maintained in Session tracking?

- By using Cookies. A Cookie is a string (in this case that string is the session ID) which is sent to a client to start a session. If the client wants to continue the session it sends back the Cookie with subsequent requests. This is the most common way to implement session tracking.
- By rewriting URLs. All links and redirections which are created by a Servlet have to be encoded to include the session ID. This is a less elegant solution (both, for Servlet implementors and users) because the session cannot be maintained by requesting a well-known URL order selecting a URL which was created in a different (or no) session.

39. What is meant by J2EE?

J2EE is the Java-centric enterprise platform specification. J2EE is used to built web sites and application around Enterprise Java Bean (EJB). Recently it has been extended to include support for XML and Web Services.

40. What are the Architecture components of J2EE?

a) Java Server Pages (JSPs: Dynamically generated Web pages hosted by the Web server. JSPs exist as text files comprised of code interspersed with HTML.

b) Struts an extension to J2EE that allows for the development of Web applications with sophisticated user-interfaces and navigation.

c) Java Servlets: These components also reside on the Web server and are used to process HTTP request and response exchanges. Unlike JSPs, servlets are compiled programs.

d) Enterprise JavaBeans (EJBs): The business components that perform the bulk of the processing within enterprise solution environments. They are deployed on dedicated application servers and can therefore leverage middleware features, such as transaction support.

41. What is meant by EJB?

EJB stands for Enterprise Java Bean and is widely-adopted server side component architecture for J2EE. It enables rapid development of mission-critical application that are versatile, reusable and portable across middleware while protecting IT investment and preventing vendor lock-in.

42. What is EJB role in J2EE?

EJB technology is the core of J2EE. It enables developers to write reusable and portable server-side business logic for the J2EE platform.

43. What are the key features of the EJB technology?

a) EJB components are server-side components written entirely in the Java programming language
b) EJB components contain business logic only – no system-level programming & services, such as transactions, security, life-cycle, threading, persistence, etc. are automatically managed for the EJB component by the EJB server.
c) EJB architecture is inherently transactional, distributed, portable multi-tier, scalable and secure.
d) EJB components are fully portable across any EJB server and any OS.
e) EJB architecture is wire-protocol neutral–any protocol can be utilized like IIOP, JRMP, HTTP, DCOM, etc.

44. What are the key benefits of the EJB technology?

1. Rapid application development
2. Broad industry adoption
3. Application portability
4. Protection of IT investment

45. How many enterprise beans?

There are three kinds of enterprise beans:

a) session beans,
b) entity beans, and
c) Message-driven beans.

46. What is EJB container?

An EJB container is a run-time environment that manages one or more enterprise beans. The EJB container manages the life cycles of enterprise bean objects, coordinates distributed transactions, and implements object security. Generally, each EJB container is provided by an EJB server and contains a set of enterprise beans that run on the server.

47. What is EJB server?

An EJB server is a high-level process or application that provides a run-time environment to support the execution of server applications that use enterprise beans. An EJB server provides a JNDI-accessible naming service, manages and coordinates the allocation of resources to client applications, provides access to system resources, and provides a transaction service. An EJB server could be provided by, for example, a database or application server.

48. What are Entity Bean and Session Bean?

- Entity Bean is a Java class which implements an Enterprise Bean interface and provides the implementation of the business methods. There are two types: Container Managed Persistence (CMP) and Bean-Managed Persistence (BMP).
- Session Bean is used to represent a workflow on behalf of a client. There are two types: ***Stateless and Stateful***. Stateless bean is the simplest bean. It doesn't maintain any conversational state with clients between method invocations. Stateful bean maintains state between invocations.

49. What is the difference between session and entity beans?

An entity bean represents persistent global data from the database; a session bean represents transient user-specific data that will die when the user disconnects (ends his session). Generally, the session beans

implement business methods (e.g.Bank.transferFunds) that call entity beans (e.g. Account.deposit, Account.withdraw).

11 MARKS

1. Explain briefly about Servlet? (5 MARKS)

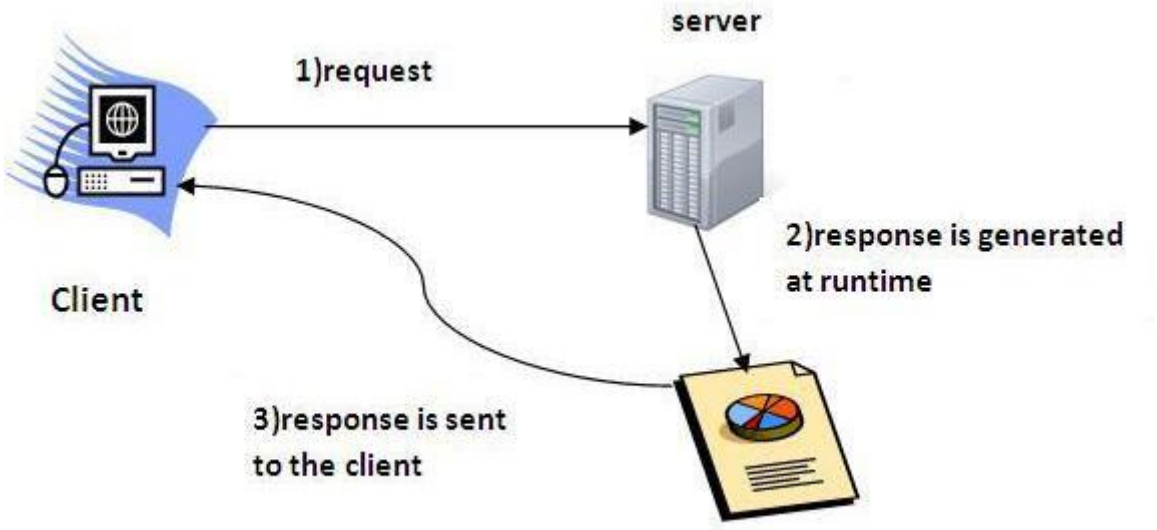
Servlet technology is used to create web application. Servlet technology is robust and scalable because of java language. Before servlet, CGI (Common Gateway Interface) scripting language was popular as a server-side programming language. But there was many disadvantages of this technology.

There are many interfaces and classes in the servlet API such as Servlet, GenericServlet, HttpServlet, ServletRequest, ServletResponse etc.

Servlet

Servlet can be described in many ways, depending on the context.

- Servlet is a technology i.e. used to create web application.
- Servlet is an API that provides many interfaces and classes including documentations.
- Servlet is an interface that must be implemented for creating any servlet.
- Servlet is a class that extend the capabilities of the servers and respond to the incoming request. It can respond to any type of requests.
- Servlet is a web component that is deployed on the server to create dynamic web page.

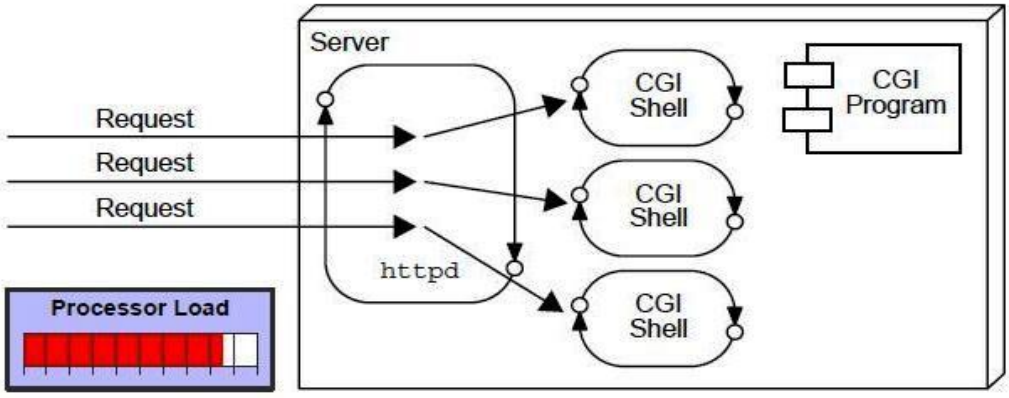


Web Application

A web application is an application accessible from the web. A web application is composed of web components like Servlet, JSP, Filter etc. and other components such as HTML. The web components typically execute in Web Server and respond to HTTP request.

CGI (Common Gateway Interface)

CGI technology enables the web server to call an external program and pass HTTP request information to the external program to process the request. For each request, it starts a new process.



Disadvantages of CGI

There are many problems in CGI technology:

1. If number of client's increases, it takes more time for sending response.
2. For each request, it starts a process and Web server is limited to start processes.
3. It uses platform dependent language e.g. C, C++, Perl.

Advantage of Servlet

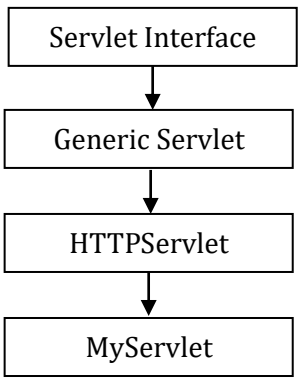
There are many advantages of servlet over CGI. The web container creates threads for handling the multiple requests to the servlet. Threads have a lot of benefits over the Processes such as they share a common memory area, lightweight, cost of communication between the threads are low.

The basic benefits of servlet are as follows:

1. **Better Performance:** because it creates a thread for each request not process.
2. **Portability:** because it uses java language.
3. **Robust:** Servlets are managed by JVM so no need to worry about memory leak, garbage collection etc.
4. **Secure:** because it uses java language.

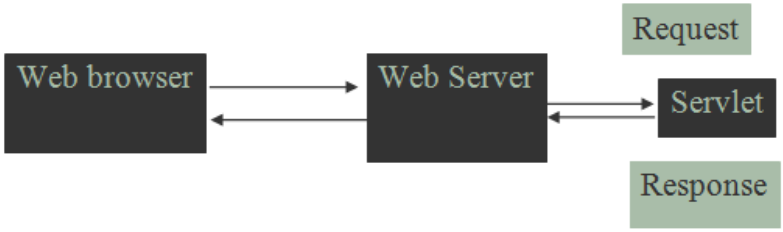
2. Explain the Architecture of Servlet in detail? (5 MARKS)

A Servlet is a class, which implements the javax.servlet.Servlet interface. However instead of directly implementing the javax.servlet.Servlet interface we extend a class that has implemented the interface like javax.servlet.GenericServletor javax.servlet.http.HttpServlet.



Servlet Execution

This is the process of a servlet execution takes place when client (browser) makes a request to the webserver.



Servlet architecture includes:

a) Servlet Interface

To write a servlet we need to implement Servlet interface. Servlet interface can be implemented directly or indirectly by extending **GenericServlet** or **HttpServlet** class.

b) Request handling methods

There are 3 methods defined in Servlet interface: **init(), service() and destroy()**.

The first time a servlet is invoked, the init method is called. It is called only once during the lifetime of a servlet.

The Service method is used for handling the client request. As the client request reaches to the container it creates a thread of the servlet object, and request and response object are also created. These request and response object are then passed as parameter to the service method, which then process the client request. The service method in turn calls the doGet or doPost methods (if the user has extended the class from HttpServlet).

c) Number of instances

Basic Structure of a Servlet

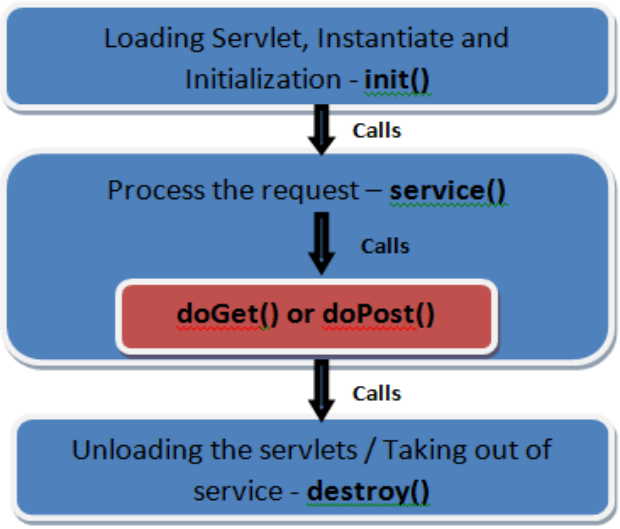
```
Public class firstServlet extends HttpServlet
{
  publicvoidinit()
  {
        /* Put your initialization code in this method,
         * as this method is called only once */
  }
  publicvoid service()
  {
        // Service request for Servlet
  }
  publicvoid destroy()
  {
      // For taking the servlet out of service, this method is called only once
  }
```

}

3. Explain the Life Cycle of Servlet? (11 MARKS)

The javax.servlet.Servlet interface defines the life cycle methods of servlet such as init(), service() and destroy(). The Web container invokes the init(), service() and destroy() methods of a servlet during its life cycle. The sequence in which the Web container calls the life cycle methods of a servlet is:

1. The Web container loads the servlet class and creates one or more instances of the servlet class.
2. The Web container invokes init() method of the servlet instance during initialization of the servlet. The init() method is invoked only once in the servlet life cycle.
3. The Web container invokes the service() method to allow a servlet to process a client request.
4. The service() method processes the request and returns the response back to the Web container.
5. The servlet then waits to receive and process subsequent requests as explained in steps 3 and 4.
6. The Web container calls the destroy() method before removing the servlet instance from the service. The destroy() method is also invoked only once in a servlet life cycle.



The init() Method

- The init() method is called during initialization phase of the servlet life cycle. The Web container first maps the requested URL to the corresponding servlet available in the Web container and then instantiates the servlet. The Web container then creates an object of the ServletConfig interface, which contains the startup configuration information, such as initialization parameters of the servlet. The Web container then calls the init() method of the servlet and passes the ServletConfig object to it.

- The init() method throws a ServletException if the Web container cannot initialize the servlet resources. The servlet initialization completes before any client requests are accepted. The following code snippet shows the init() method:

```
Public class ServletLifeCycle extends HttpServlet
{
```

```
        static int count;
        public void init (ServletConfig config) throws ServletException
        {
                count=0;
        }
    }
```

The service() Method

The service() method processes the client requests. Each time the Web container receives a client request, it invokes the service() method. The service() method is invoked only after the initialization of the servlet is complete. When the Web container calls the service() method, it passes an object of the ServletRequest interface and an object of the ServletResponse interface. The ServletRequest object contains information about the service request made by the client. The ServletResponse object contains the information returned by the servlet to the client.

The following code snippet shows the service() method:

```
public void service(ServletRequest req, ServletResponse res) throws ServletException,
IOException
```

The service() method throws ServletException exception when an exception occurs that interferes with the normal operation of the servlet. The service() method throws IOException when an input or output exception occurs.

The service() method dispatches a client request to one of the request handler methods of the HttpServlet interface, such as the doGet(), doPost(), doHead() or doPut(). The request handler methods accept the objects of the HttpServletRequest and HttpServletResponse as parameters from the service() method.

The doGet() Method

The doGet() method processes client request, which is sent by the client, using the HTTP GET method. GET is a type of HTTP request method that is commonly used to retrieve static resources. To handle client requests that are received using GET method, we need to override the doGet() method in the servlet class. In the doGet() method, we can retrieve the client information of the HttpServletRequest object. We can use the HttpServletResponse object to send the response back to the client.

The following code snippet shows the doGet() method:

```
public void doGet (HttpServletRequest req , HttpServletResponse res) throws ServletException,
```

IOException

The doPost() Method

The doPost() method handles requests in a servlet, which is sent by the client, using the HTTP POST method. For example, if a client is entering registration data in an HTML form, the data can be sent using the POST method. Unlike the GET method, the POST request sends the data as part of the HTTP request body. As a result, the data sent does not appear as a part of URL.

To handle requests in a servlet that is sent using the POST method, we need to override the doPost() method. In the doPost() method, we can process the request and send the response back to the client.

The following code snippet shows the doPost() method:

```
public void doPost (HttpServletRequest req , HttpServletResponse res) throws ServletException,
IOException
```

The doHead() Method

The doHead() method handles requests sent using the HTTP HEAD method. Similar to the GET method, the HEAD method also sends requests to the server. The only difference between the GET and the HEAD methods is that the HEAD method returns the header of the response, which contains entries, such as Content-Type, Content-Length, and Last-Modified. The HEAD method is used to find whether a requested resource exists. It is also used to obtain information about the resource, such as type of the resource. This information is required before requesting for resource content.

The following code snippet shows the doHead() method:

```
public void doHead (HttpServletRequest req , HttpServletResponse res) throws ServletException,
IOException
```

The doPut() Method

The doPut() method handles requests sent using the HTTP PUT method. The PUT method allows a client to store information on the server.

The following code snippet shows the doPut() method:

```
Public void doPut (HttpServletRequest req , HttpServletResponse res) throws ServletException,
IOException
```

The destroy() Method

The destroy() method marks the end of the life cycle of a servlet. The Web container calls the destroy() method before removing a servlet instance from the service.

The Web container calls the destroy() method as

- The time period specified for the servlet has elapsed. The time period of a servlet is the period for which the servlet is kept in the active state by the Web container to service the client request.
- The Web container needs to release servlet instances to conserve memory.
- The Web container is about to shut down.

In the destroy() method we can write the code to release the resources occupied by the servlet. The destroy() method is also used to save any persistent information before the servlet instance is removed from the service.

The following code snippet shows the destroy() method:

```
public void destroy();
```

4. Explain Generic Servlet and HTTP Servlet in detail? (11 MARKS)

Generic Servlet:

GenericServlet class defines a protocol-independent (HTTP-less) servlet. However while building a website or an online application, you may want to have Http protocol, in that case extend HttpServlet instead of GenericServlet class.

The blue print of GenericServlet class:

```
public abstract class GenericServlet extends java.lang.Object implements Servlet,
ServletConfig,java.io.Serializable
```

An abstract class and it extends Object class & implements Servlet, ServletConfig and Serializable interfaces.

Methods of Generic Servlet class

- **void destroy()**: It is called by servlet container to indicate that the servlet life cycle is finished.
- **java.lang.String getInitParameter (java.lang.String name)**: It returns the value of the specified initialization parameter. If the parameter does not exist then it returns null.
- **java.util.Enumeration getInitParameterNames()**: This method returns all the initialization parameters in form of a String enumeration. By using enumeration methods we can fetch the individual parameter names.
- **ServletConfig getServletConfig()**: It returns this servlet's ServletConfig object.
- **ServletContext getServletContext()**: It returns a reference to the ServletContext in which this servlet is running.
- **java.lang.StringgetServletInfo()**: It returns information about the servlet, such as author, version, and copyright.
- **java.lang.StringgetServletName()**: This method returns the name of this servlet instance.
- **void init()**: This is the first method of servlet life cycle and it is used for initializing a servlet.
- **void init(ServletConfig config)**: Called by the servlet container to indicate to a servlet that the servlet is being placed into service.
- **void log (java.lang.Stringmsg)**: Writes the specified message to a servlet log file, prefixed with servlet's name.
- **void log (java.lang.String message, java.lang.Throwable t)**
- **abstract void service (ServletRequest req, ServletResponse res)**: It is called by the servlet container to allow the servlet to respond to the client's request.

Example of Generic Servlet

index.html

```
<a href="welcome"> Click to call Servlet </a>
```

ExampleGeneric.java

```
import java.io.*;
import javax.servlet.*;
public class ExampleGeneric extends GenericServlet
{
  public void service(ServletRequest req , ServletResponse res) throws IOException,
  ServletException
  {
        res.setContentType("text/html");
        PrintWriterpwriter=res.getWriter();
        pwriter.print("<html>");
        pwriter.print("<body>");
        pwriter.print("<h2> Generic Servlet Example </h2>");
        pwriter.print("</body>");
        pwriter.print("</html>");
  }
}
```

web.xml

```
<web-app>
   <servlet>
        <servlet-name> Beginners book </servlet-name>
        <servlet-class> /ExampleGeneric </servlet-class>
  </servlet>
   <servlet-mapping>
        <servlet-name> Beginners book </servlet-name>
        <url-pattern> welcome </url-pattern>
  </servlet-mapping>
</web-app>
```

HTTP Servlet:

The Http Servlet class implements Serializable interface and extends Generic Servlet class. It provides the definition of HTTP specific methods such as doGet(), doPost(), doTrace() etc.

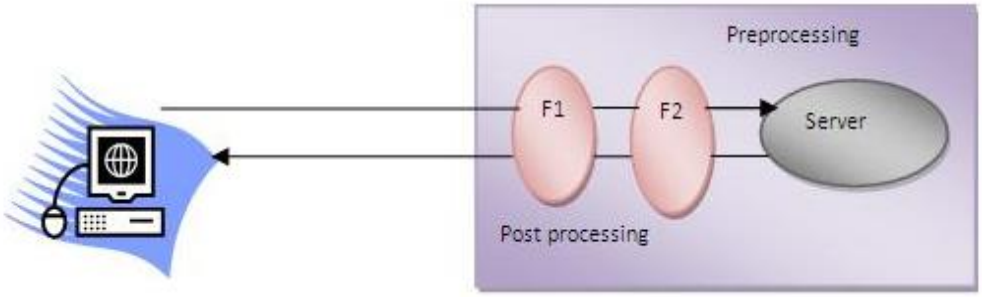
Methods

The methods of HttpServlet class are as follows:

1. **protected void doGet (HttpServletRequest request, HttpServletResponse response)**: It handles the GET request. It is invoked by the web container.
2. **protected void doPost (HttpServletRequest request, HttpServletResponse response)**: Similar to doGet() method, it also gets invoked by the web container and it handles the POST request.
3. **public void service (ServletRequest request , ServletResponse response)**: There are two types of service method, one is public and other one is protected. The purpose of this public method is to dispatch the request to the protected service method by converting the request and response object into HTTP type.
4. **protected void service (HttpServletRequest request , HttpServletResponse response)**: This method receives the request from the public service() method. It dispatches the request to the doXXX() method depending on the incoming http request type.
5. **protected void doHead (HttpServletRequest request , HttpServletResponse response)**: It handles the HEAD request.
6. **protected void doOptions (HttpServletRequest request , HttpServletResponse response)**: Performs the HTTP OPTIONS operation; the default implementation of this method automatically determines what HTTP Options are supported.
7. **protected void doPut (HttpServletRequest request , HttpServletResponse response)**: Performs the HTTP PUT operation; the default implementation reports an HTTP BAD_REQUEST error.
8. **protected void doTrace (HttpServletRequest request , HttpServletResponse response)**: Performs the HTTP TRACE operation; the default implementation of this method causes a response with a message containing all of the headers sent in the trace request.
9. **protected void doDelete (HttpServletRequest request , HttpServletResponse response)**: Performs the HTTP DELETE operation; the default implementation reports an HTTP BAD_REQUEST error.
10. **protected long getLastModified (HttpServletRequest request)**: Gets the time the requested entity was last modified; the default implementation returns a negative number, indicating that the modification time is unknown and hence should not be used for conditional GET operations or for other cache control operations as this implementation will always return the contents.

5. Explain briefly about Filter Concepts? (11 MARKS)

A filter is an object that is invoked at the preprocessing and post processing of a request. It is mainly used to perform filtering tasks such as conversion, logging, compression, encryption and decryption, input validation etc. The servlet filter is pluggable, i.e. its entry is defined in the web.xml file, if we remove the entry of filter from the web.xml file, filter will be removed automatically and we don't need to change the servlet. So maintenance cost will be less.



Usage of Filter

- Recording all incoming requests
- Logs the IP addresses of the computers from which the requests originate
- Conversion
- Data compression
- Encryption and decryption
- Input validation etc

Advantage of Filter

- Filter is pluggable.
- One filter don't have dependency onto another resource.
- Less Maintenance

Filter API

The servlet filter have its won API. The javax.servlet package contains the three interfaces of Filter API.

1. Filter
2. FilterChain
3. FilterConfig

1) Filter interface

For creating any filter, you must implement the Filter interface. Filter interface provides the life cycle methods for a filter.

Method	Description
public void init (FilterConfig config)	init() method is invoked only once. It is used to initialize the filter.
public void doFilter (HttpServletRequest request, HttpServletResponse response, FilterChain chain)	doFilter() method is invoked every time when user request to any resource, to which the filter is mapped.It is used to perform filtering tasks.
public void destroy()	This is invoked only once when filter is taken out of the service.

2) FilterChain interface

The object of FilterChain is responsible to invoke the next filter or resource in the chain. This object is passed in the doFilter method of Filter interface. The FilterChain interface contains only one method:

public void doFilter(HttpServletRequest request, HttpServletResponse response): it passes the control to the next filter or resource.

How to define Filter

We can define filter same as servlet. Let's see the elements of filter and filter-mapping.

```
<web-app>
  <filter>
      <filter-name> ... </filter-name>
      <filter-class> ... </filter-class>
  </filter>
  <filter-mapping>
      <filter-name> ... </filter-name>
      <url-pattern> ... </url-pattern>
  </filter-mapping>
</web-app>
```

For mapping filter we can use, either url-pattern or servlet-name. The url-pattern elements has an advantage over servlet-name element i.e. it can be applied on servlet, JSP or HTML.

Example of Filter

We are simply displaying information that filter is invoked automatically after the post processing of the request.

index.html

```
<a href="servlet1"> click here </a>
```

MyFilter.java

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.*;
public class MyFilter implements Filter
{
public void init(FilterConfig arg0) throws ServletException {}
public void doFilter(ServletRequest req, ServletResponse resp,  FilterChain chain) throws IOException,
ServletException
{
        PrintWriter out=resp.getWriter();
        out.print("filter is invoked before");
        chain.doFilter(req, resp);                    //sends request to next resource
         out.print("filter is invoked after");
}
```

```
 public void destroy() {}
}
```

HelloServlet.java

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.*;
public class HelloServlet extends HttpServlet
 {
public void doGet(HttpServletRequest request, HttpServletResponse response)throws ServletException,
IOException
 {
            response.setContentType("text/html");
            PrintWriter out = response.getWriter();
            out.print("<br> welcome to servlet <br>");
}
}
```

web.xml

For defining the filter, filter element of web-app must be defined just like servlet.

```
 <web-app>
   <servlet>
            <servlet-name> s1 </servlet-name>
            <servlet-class> Hello Servlet </servlet-class>
   </servlet>
   <servlet-mapping>
            <servlet-name> s1 </servlet-name>
            <url-pattern>/servlet1</url-pattern>
   </servlet-mapping>
   <filter>
            <filter-name> f1 </filter-name>
            <filter-class> MyFilter </filter-class>
   </filter>
   <filter-mapping>
            <filter-name> f1 </filter-name>
            <url-pattern> /servlet1 </url-pattern>
            </filter-mapping>
 </web-app>
```

6. Discuss basic concepts in Java Server Pages (JSP)? (6 MARKS)

- JSP abbreviated as Java Server Pages.
- JSP is Sun's solution for developing dynamic web sites.
- JSP provide excellent server side scripting support for creating database driven web applications.
- JSP is a server side technology that enables web programming to generate web pages dynamically in response to client requests.
- JSP uses HTML, XML and Java code, the application are secure, fast and independent of server platforms.
- JSP is nothing but high level abstraction of java servlet technology.
- It allows us to directly embed pure java code in an HTML page.
- JSP pages run under the supervision of an environment called web container.
- The web container compiles the page on the server and generates a servlet, which is loaded in the Java Runtime Environment (JRE).
- JSP enables the developers to directly insert java code into JSP file.
- JSP pages are efficient, they load into the web server's memory on receiving the request at the very first time and the subsequent calls are served within a very short period of time.
- This makes the development of the web applications convenient, simple and fast.
- Maintenance also becomes very easy.
- In today's environment, most website server's dynamic pages are based on user request.
- Database is very convenient way to store the data of users.
- JSP file have the extension .jsp

JSP and ASP

- JSP is portable or running the entire platform. Asp runs only in MS windows platform.
- Similar operation

JSP and JavaScript

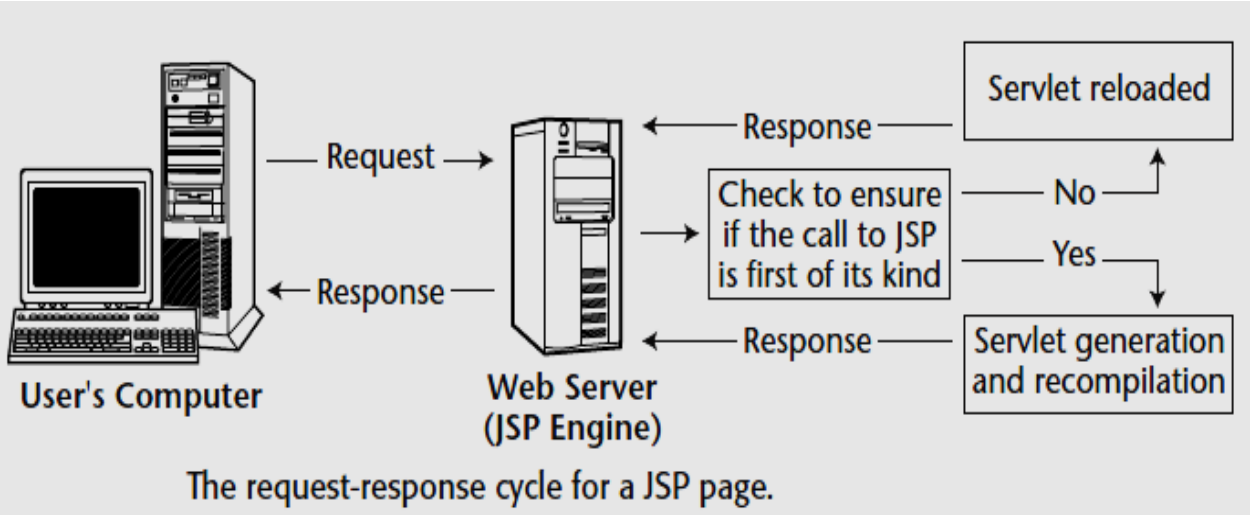
JSP

- ✓ It dynamically Web Pages can be changed on the server.
- ✓ It can access server side resources like database, catalogs, pricing information.
- ✓ It can access session tracking and cookies.

JavaScript

- ✓ It can generate dynamically on the client side.
- ✓ It does not access this type of resources.
- ✓ It does not have.

JSP working Model:



The request-response cycle for a JSP page.

JSP Engines

- ✓ To Process JSP page, a JSP engine is needed.
- ✓ JSP Engine is nothing but a specialized servlet, which runs under the supervision of the servlet engine.
- ✓ The JSP Engine is typically connected with a web server or can be integrated inside a web server or an application server.

Many servers are

1. Tomcat
2. Java Web Server
3. WebLogic
4. WebSphere

Life Cycle of a Servlet and JSP

- A web server is responsible for initializing, invoking and destroying.
- A web server communicates with a servlet through a simple interfaces as
  1. JspInit()
  2. JspService()
  3. JspDestroy()

How JSP works?

1. Translates the JSP source code into the servlet source code.
2. Complies the servlet source code to generate a class file.
3. Loads the class file and create an instance.
4. Initializes the servlet instance by calling the jspInit() method.
5. Invokes the jspService() method, passing the request and response objects.

7. Explain the components of JSP in detail? **(11 MARKS)**

JSP Syntax:

The syntax of JSP is almost similar to that of XML.

The general rules are

1. Tags must have their matching end tags.
2. Attributes must appear in the start tags.
3. Attribute values in the tag must be quoted.

Components of JSP:

JSP page is built using components such as

1. Directives
2. Scripting Elements
   - ✓ Expressions
   - ✓ Declarations
   - ✓ Scriptlets
3. Standard Actions
4. Custom Tags

1. Directives

- A Directives element in a JSP page provides global information amount a particular JSP page.
- The syntax of jsp directive is

  <%@ directive-name attribute="value" %>

The three standard directives available in all JSP environments:

1. Page
2. include
3. Taglib

Page Directives:

- The page directive defines attributes that notify the web container about the general settings of a JSP page.
- It has the following syntax:

  <%@ page attribute="value" attribute="value" %>

Example:

<%@ page language="java" %>

<%@ page language="java" import="java.sql.*" %>

Page Directives Attributes

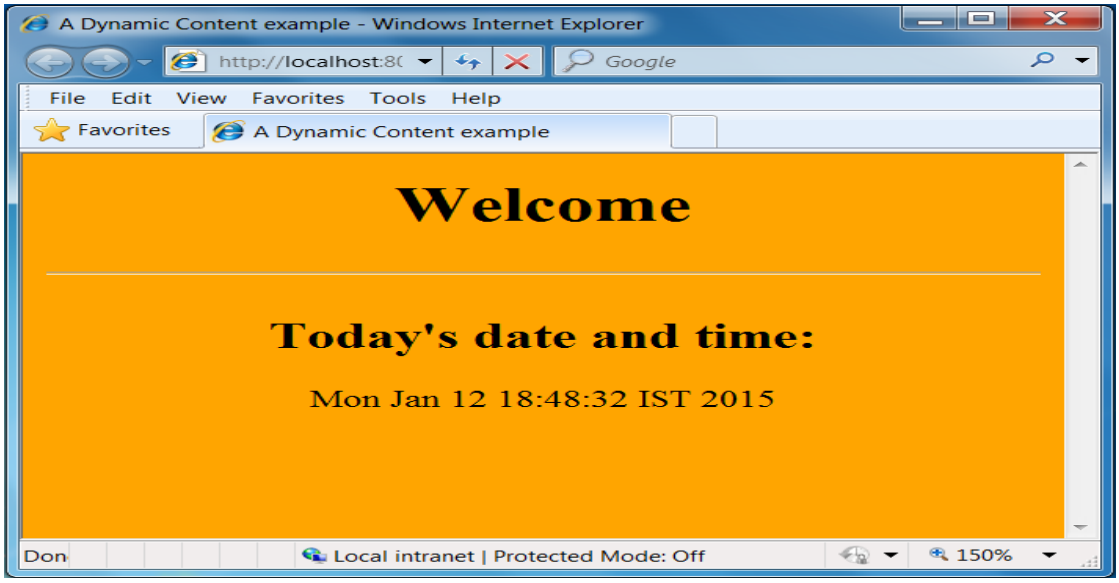
Attribute	Value
`language`	The language used in scriptlets, expressions, and declarations. In JSP 1.1, the only valid value for this attribute is java.
`extends`	The fully qualified name of the superclass of this JSP page. This must be a class that implements the `HttpJspPage` interface. The JSP specification warns against the use of this attribute without fully understanding its implications.
`import`	A comma-separated list of one or more *package.** names and/or fully qualified class names. This list is used to create corresponding import statements in the generated Java servlet. The following packages are automatically included and need not be specified: java.lang.* java.servlet.* java.servlet.jsp.* java.servlet.http.*
`session`	*true* or *false*, indicating whether the JSP page requires an HTTP session. If the value is true, then the generated servlet will contain code that causes an HTTP session to be created (or accessed, if it already exists). The default value is *true*.
`buffer`	Specifies the size of the output buffer. Valid entries are *nnn*kb or none, where *nnn* is the number of kilobytes allocated for the buffer. The default value is 8kb.
`autoflush`	*true* if the buffer should be automatically flushed when it is full, or *false* if a buffer overflow exception should be thrown. The default value is *true*.

Attribute	Value
`isThreadSafe`	*true* if the page can handle simultaneous requests from multiple threads, or *false* if it cannot. If *false*, the generated servlet declares that it implements the `SingleThreadModel` interface.
`info`	A string that will be returned by the page's `getServletInfo()` method.
`isErrorPage`	*true* if this page is intended to be used as another JSP's error page. In that case, this page can be specified as the value of the `errorPage` attribute in the other page's page directive. Specifying *true* for this attribute makes the *exception* implicit variable available to this page. The default value is *false*.
`errorPage`	Specifies the URL of another JSP page that will be invoked to handle any uncaught exceptions. The other JSP page must specify `isErrorPage="true"` in its page directive.
`contentType`	Specifies the MIME type and, optionally, the character encoding to be used in the generated servlet.

Page Directives Examples

```
<%@ page language="java" %>
<html>
        <head>
                <title> Simple JSP example </title>
        </head>
        <body>
                <h1> Welcome </h1> <hr>
                <h3>Today's date and time: </h3>
        <%-- This is the JSP content that displays the server time by using the method Date() --%>
                <% =new java.util.Date() %>
        </body>
</html>
```

Output



Include Directives

- Include directives are too used to specify the name of the files to be inserted during the compilation of the JSP page.
- This directive is to include the HTML, JSP or Servlet file into a JSP file.
- This is a static inclusion to a file.

Syntax

<%@ include file="filename" %>

Example

```
<%@ include file="/header.html" %>
<%@ include file="/doc/legal/disclaimer.html" %>
<%@ include file="sortmethod" %>
```

Include Directives Examples

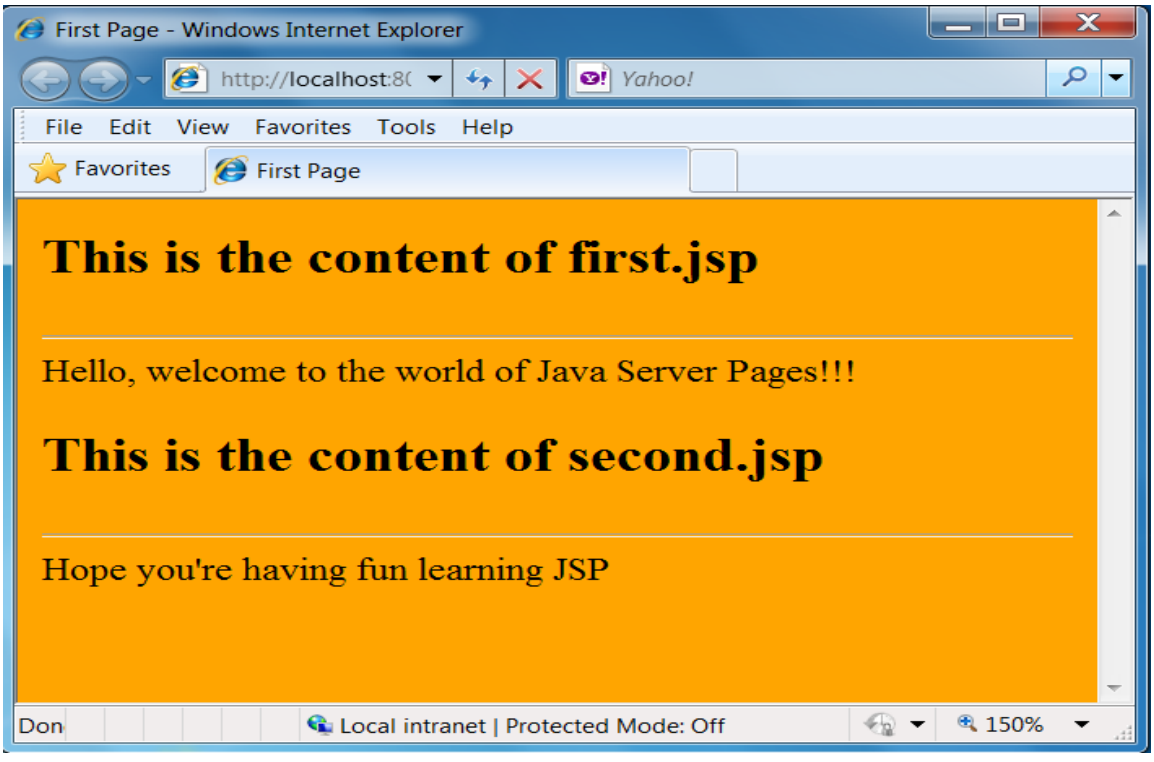
first.jsp

```
<html>
        <head>
                <title> An Include example </title>
        </head>
        <body>
                <h2> This is the content of first.jsp </h2> <hr>
                <% =”Hello, welcome to the world of Java Server Pages!!!” %>
                <%@ include file=”second.jsp” %>
        </body>
</html>
```

Second.jsp

```
<html>
        <head>
                <title> An Include example </title>
        </head>
        <body>
                <h2> This is the content of second. Jsp </h2> <hr>
                <%= “Hope you’re having fun learning JSP” %>
        </body>
</html>
```

Output



Comments

- The JSP comment syntax is

  <%- - This is a hidden JSP comment - - %>

- and the latter looks like this:

  <! - - This is included in the generated HTML - ->

2. Scripting Elements

The 3 different types of Scripting Elements are

1. Expressions
2. Declarations
3. Scriptlets

Expressions

Syntax of JSP Expression is

<% = expression %>

- JSP Expression start with <% = and ends with %>.
- Between these, any number of expressions can be embedded.
- The result of expression is converted to the string to get displayed.

Example

<%="Hello World!" %>

Expression examples

```
<HTML>
        <HEAD>
                <TITLE> My first JSP Example </TITLE>
        </HEAD>

        <BODY>
                <H1> My first JSP example </H1> <HR>
                <%-- This is the JSP content --%>
                <%="Hello World" %>
        </BODY>
</HTML>
```

Declarations:

The syntax of JSP Declaratives is

```
<%!
   // declare all the variables here
%>
```

- JSP declaration begins with <%! and ends with %>.
- We can embed any amount of java code in the JSP declarations.
- Variables and functions in the declarations are class level and can be used anywhere in the JSP page.

Scriptlets:

The Syntax of JSP Scriptlets

```
<%
    //java code
%>
```

Example:

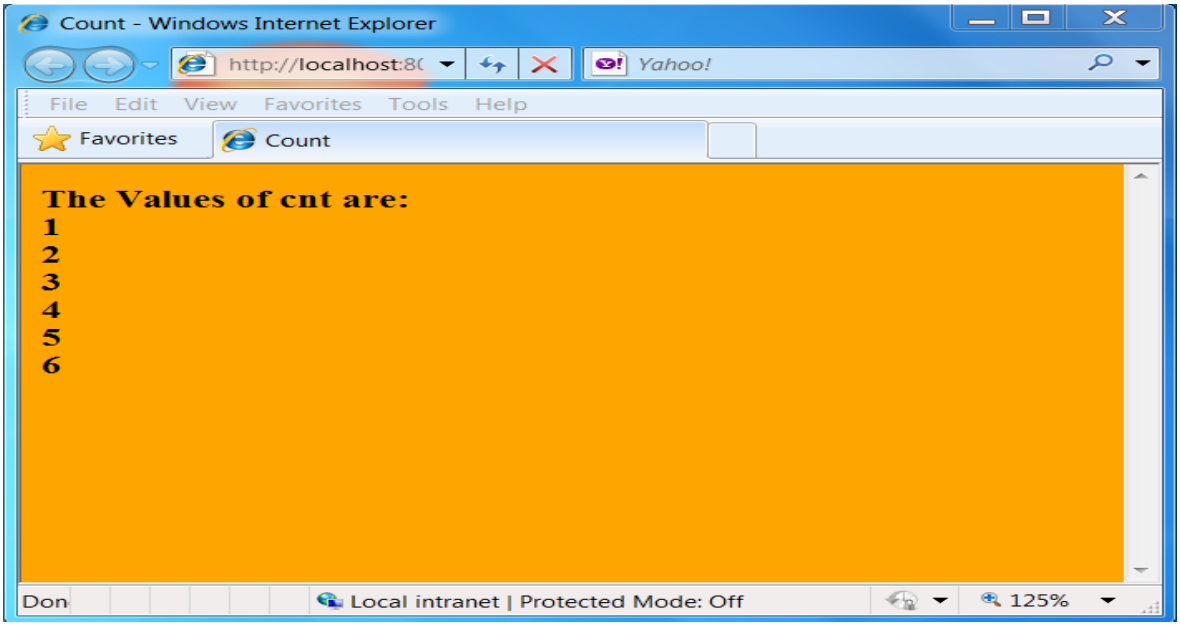
```
<%
    String username=cse;
    Username = request.getParameter("username");
%>
```

Program 1:

```
<%@ page language="java"%>
<html>
    <head>
        <title> Count program in JSP </title>
    </head>
    <body>
        <%!
            intcnt=0;
            privateintgetcount()
            {
                // increment cnt and return the value
                cnt++;
                returncnt;
            }
        %>
        <p> The Values of cnt are:
        <%=getcount()%>
        <%=getcount()%>
```

```
                <%=getcount()%>
                <%=getcount()%>
        </body>
    </html>
```

Output



- We can embed any amount of java code in the JSP Scriptlets.
- JSP Engine places these codes in JspService() methods.
- Variables available to the JSP Scriptlets are
    1. **Request** – HttpServletRequest
    2. **Response** – HttpServletResponse
    3. **Session** – HTTP session object associated with the request
    4. **Out** - is an object of output stream and is used to send any output to the client.

3. Standard Action:

- Actions are high-level JSP elements that create, modify, or use other objects.
- The syntax for the JSP standard Action is

    <tagname [attr="value" attr="value" ...] >

    ...

    </tag-name>

The various Tag name are

1. Include
2. Forward
3. Param
4. Plugin

Include:

- The Inclusion of file is dynamic and the specified file is included in the JSP file at runtime.
- Output of the included file is inserted into the JSP file.

Syntax:

<jsp: include page =“filename”>

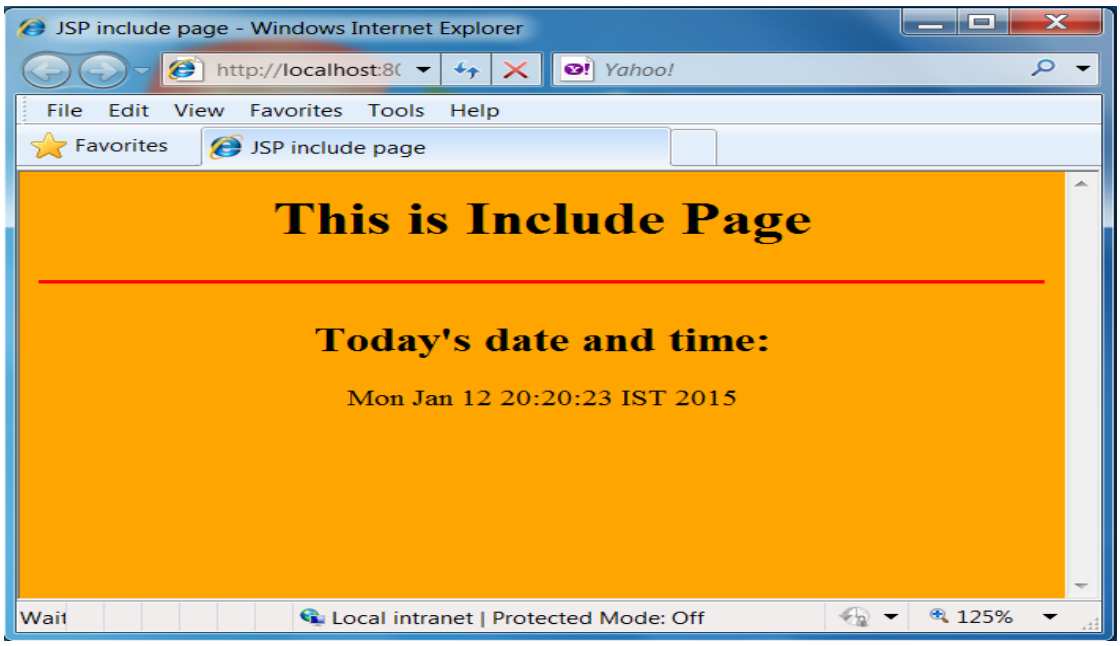
Include Example

```
<%@page language="java"%>
<html>
<head>
        <title> JSP include page </title>
</head>
<body>
        <%-- using the <jsp:include> action --%>
        <h1 align="center"> This is Include Page </h1> <hr color="red">
        <jsp:include page="dateutil.jsp" flush="true"/>
</body>
</html>
```

dateutil.jsp

```
<%@ page language="java"%>
<html>
<head>
        <title> A Dynamic Content example </title>
</head>
<body bgcolor="orange">
    <center>
        <h2> Today's date and time: </h2>
        <%= new java.util.Date() %>
    </center>
</body>
</html>
```

Output:



Forward

This will redirect to the different page without notifying browser.

Syntax:

<jsp: forward page =“filename”/>

Param and Plugin:

`<jsp:param>`	Binds a value to a name and passes the binding to another resource invoked with `<jsp:include>` or `<jsp:forward>`. Syntax is <jsp:param name="*name*" value="*value*" />
`<jsp:plugin>`	Used to generate the appropriate HTML linkage for downloading the Java plugin: <jsp:plugin type="*bean* | *applet*" code="*objectCode*" codebase="*objectCodebase*" { align="*alignment*" } { archive="*archiveList*" } { height="*height*" } { hspace="*hspace*" } { jreversion="*jreversion*" } { name="*componentName*" } { vspace="*vspace*" } { width="*width*"} { nspluginurl="*url*" } { iepluginurl="*url*" } > { <jsp:params> { <jsp:param name="*name*" value="*value*" /> }+</jsp:params> }}</jsp:plugin>

Custom Tags

- The taglib directive makes custom actions available in the current page through the use of a tag library.

The syntax of the directive is

<%@ tagliburi="tagLibraryURI" prefix="tagPrefix" %>

Attribute	Value
tagLibraryURI	The URL of a Tag Library Descriptor.
tagPrefix	A unique prefix used to identify custom tags used later in the page.

Implicit Objects

Variable Name	Value
request	The ServletRequest or HttpServletRequest being serviced.
response	The ServletResponse or HttpServletResponse that will receive the generated HTML output.
pageContext	The PageContext object for this page. This object is a central repository for attribute data for the page, request, session, and application.
session	If the JSP page uses an HttpSession, it is available here under the name session.
application	The servlet context object.

Variable Name	Value
out	The character output stream used to generate the output HTML.
config	The ServletConfig object for this servlet context.
page	A reference to the JSP page itself.
exception	An uncaught exception that causes the error page to be invoked. This variable is available only to pages with isErrorPage="true".

8. Discuss about the Session and Cookies? (11 MARKS)

Session Tracking:

J2EEEis a mechanism that servlets use to maintain state about a series of requests from the same user (that is, requests originating from the same browser) across some period of time.

Session-Tracking basics

Every user of a site is associated with a javax.servlet.http.HttpSession object that servlets can use to store or retrieve information about that user

Basic session tracking functionalities,

- ✓ Obtain a session (an **HttpSession** object) for a user.
- ✓ Store or get data from the HttpSession object.
- ✓ Invalidate the session (optional).

Get Session

- A servlet uses its request object's getSession() method to retrieve the current HttpSession object.
- public HttpSessionHttpServletRequest.getSession(boolean create)
- This method returns the current session associated with the user making the request.
- If the user has no current valid session, this method creates one if create is true or returns null if create is false.
- To ensure the session is properly maintained, this method must be called at least once before any output is written to the response.

Put Value

- You can add data to an HttpSession object with the putValue() method:
- public void HttpSession.putValue(String name, Object value)
- This method binds the specified object value under the specified name. Any existing binding with the same name is replaced.

Get Value

- To retrieve an object from a session, use getValue():
- public Object HttpSession.getValue(String name)
- This method returns the object bound under the specified name or null if there is no binding.
- You can also get the names of all of the objects bound to a session with
- getValueNames():
- public String[] HttpSession.getValueNames()
- This method returns an array that contains the names of all objects bound to this session or an empty (zero length) array if there are no bindings.

Remove Value

- Finally, you can remove an object from a session with removeValue():
- public void HttpSession.removeValue(String name)
- This method removes the object bound to the specified name or does nothing if there is no binding
- Each of these methods can throw a java.lang.IllegalStateException if the session being accessed is invalid.

The Session Life Cycle

- Sessions do not last forever.
- A session either expires automatically, after a set time of inactivity (for the Java Web Server the default is 30 minutes), or manually, when it is explicitly invalidated by a servlet.
- When a session expires (or is invalidated), the HttpSession object and the data values it contains are removed from the system.
- Beware that any information saved in a user's session object is lost when the session is invalidated.

- If you need to retain information beyond that time, you should keep it in an external location (such as a database) and store a handle to the external data in the session object (or your own persistent cookie)

URL Rewriting

The Methods involved in managing the session life cycle

1. **Public boolean HttpSession.isNew() -** This method returns whether the session is new or not.
2. **public void HttpSession.invalidate() -**This method causes the session to be immediately invalidated. All objects stored in the session are unbound.
3. **public long HttpSession.getCreationTime() -** This method returns the time at which the session was created.
4. **public long HttpSession.getLastAccessedTime() -** This method returns the time at which the client last sent a request associated with this session.

COOKIES

Cookies are short pieces of data sent by web servers to the client browser. The cookies are saved to client's hard disk in the form of small text file. Cookies help the web servers to identify web users, by this way server tracks the user. Cookies pay very important role in the session tracking.

Cookie Class

- In JSP cookie are the objects of the class **javax.servlet.http.Cookie**. This class is used to create a cookie, a small amount of information sent by a servlet to a Web browser, saved by the browser, and later sent back to the server. A cookie's value can uniquely identify a client, so cookies are commonly used for session management. A cookie has a name, a single value, and optional attributes such as a comment, path and domain qualifiers, a maximum age, and a version number.
- The getCookies() method of the request object returns an array of Cookie objects. Cookies can be constructed using the following code:

Cookie(java.lang.String name, java.lang.String value)

Cookie objects have the following methods.

Method	Description
getComment()	Returns the comment describing the purpose of this cookie, or null if no such comment has been defined.
getMaxAge()	Returns the maximum specified age of the cookie.
getName()	Returns the name of the cookie.
getPath()	Returns the prefix of all URLs for which this cookie is targeted.
getValue()	Returns the value of the cookie.
setComment(String)	If a web browser presents this cookie to a user, the cookie's purpose will be described using this comment.
setMaxAge(int)	Sets the maximum age of the cookie. The cookie will expire after that many

	seconds have passed. Negative values indicate the default behavior: the cookie is not stored persistently, and will be deleted when the user web browser exits. A zero value causes the cookie to be deleted
setPath(String)	This cookie should be presented only with requests beginning with this URL.
setValue(String)	Sets the value of the cookie. Values with various special characters (white space, brackets and parentheses, the equals sign, comma, double quote, slashes, question marks, the "at" sign, colon, and semicolon) should be avoided. Empty values may not behave the same way on all browsers.

Example Using Cookies

To write code in JSP file to set and then display the cookie.

Create Form

Here is the code of the form (cookieform.jsp) which prompts the user to enter his/her name.

```
<%@ page language="java" %>
<html>
        <head>
                <title> Cookie Input Form </title>
        </head>
        <body>
                <form method="post" action="setcookie.jsp">
                    <p> <b> Enter Your Name: </b> <input type="text" name="username"> <br>
                   <input type="submit" value="Submit">
                </form>
        </body>
</html>
```

Above form prompts the user to enter the user name. User input are posted to the setcookie.jsp file, which sets the cookie. Here is the code of **setcookie.jsp file:**

```
<%@ page language="java" import="java.util.*"%>
        <%
                String username=request.getParameter("username");
                if(username==null) username="";
                Date now = new Date();
                String timestamp = now.toString();
                Cookie cookie = new Cookie ("username",username);
                cookie.setMaxAge(365 * 24 * 60 * 60);
                response.addCookie(cookie);
        %>
<html>
```

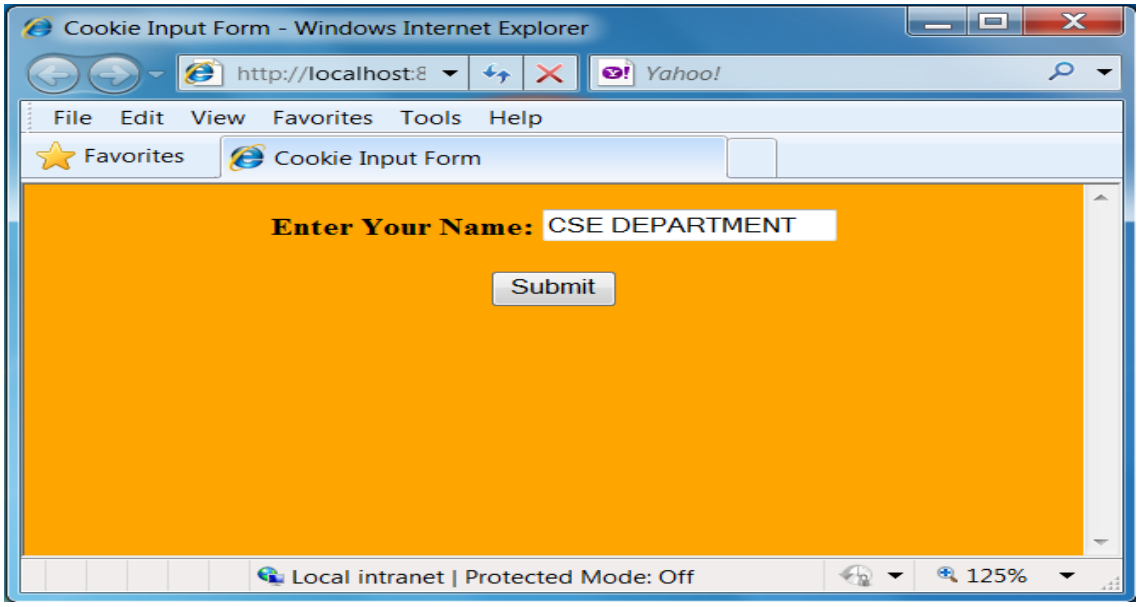
```
        <head>
                <title> Cookie Saved </title>
        </head>
        <body>
                <p> <a href="showcookievalue.jsp"> Next Page to view the cookie value </a> <p>
        </body>
</html>
```

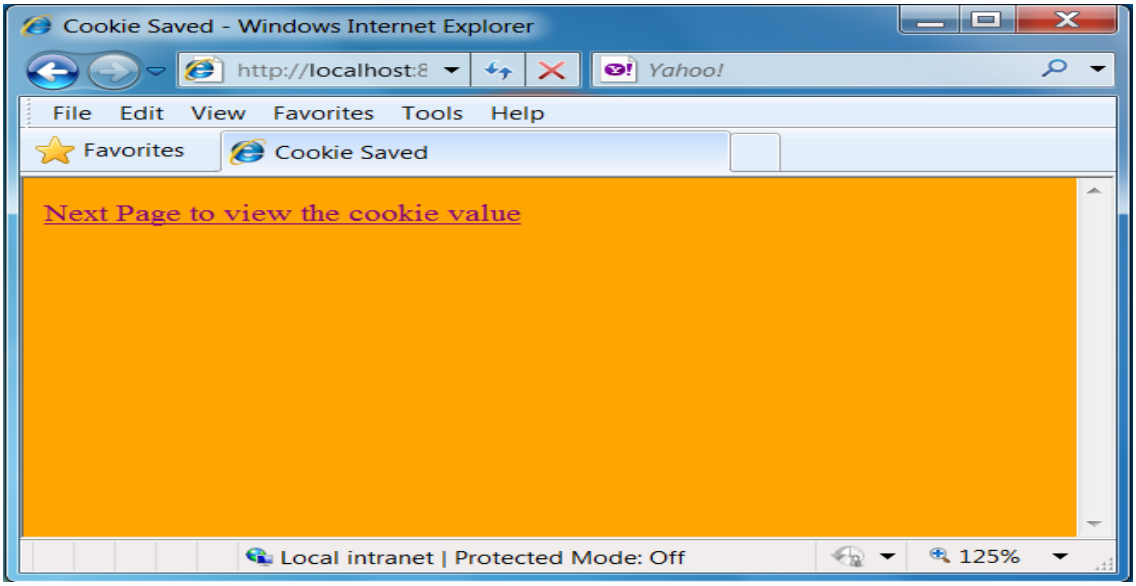
Above code sets the cookie and then displays a link to view cookie page. Here is the code of display cookie page **(showcookievalue.jsp):**

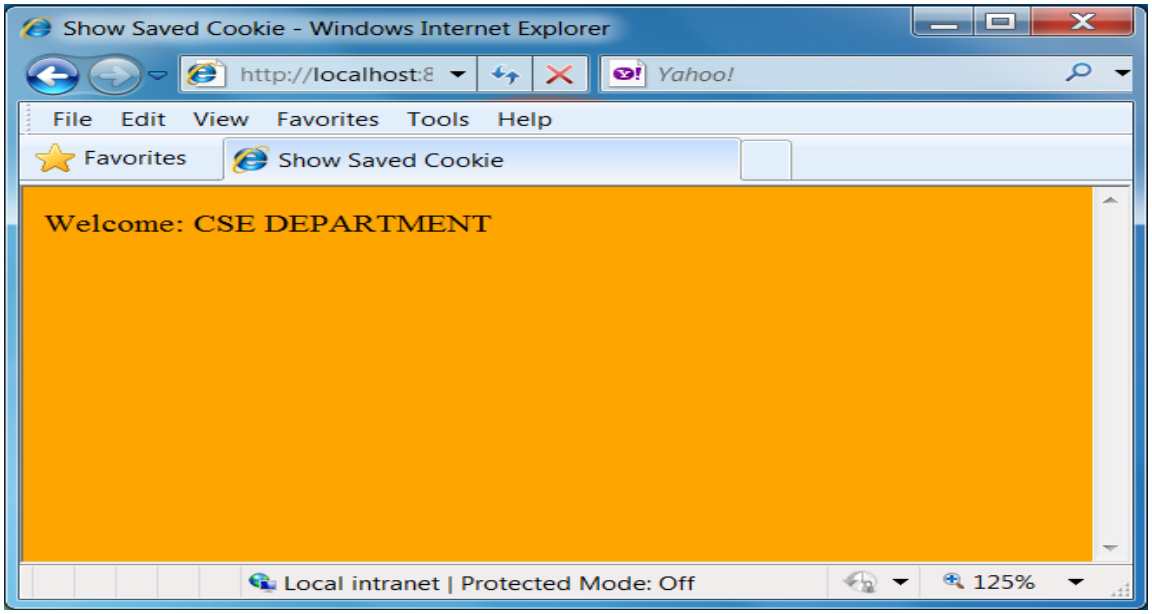
```
<%@ page language="java" %>
        <%
                String cookieName = "username";
                Cookie cookies [] = request.getCookies ();
                Cookie myCookie = null;
                if (cookies != null)
                {
                        for (inti = 0; i<cookies.length; i++)
                        {
                                if (cookies [i].getName().equals (cookieName))
                                {
                                        myCookie = cookies[i];
                                        break;
                                }
                        }
                }
        %>
<html>
        <head>
                <title>Show Saved Cookie</title>
        </head>
        <body>
                <%
                        if (myCookie == null)
                        {
                %>
                        No Cookie found with the name <%=cookieName%>
                <%
                        }
```

```
                else
                {
        %>
                <p>Welcome: <%=myCookie.getValue()%>.
        <%
                }
        %>
</body> </html>
```

When user navigates to the above the page, cookie value is displayed.







9. Explain how to Access a database from JSP? **(11 MARKS)**

The connectivity from MYSQL database with JSP. We take an example of **Books** database. This database contains a table named **books_details**. This table contains three fields- **id**, **book_name**&**author**. We start from very beginning. First we learn how to create tables in MySQl database after that we write a html page for inserting the values in '**books_details**' table in database. After submitting values a table will be showed that contains the book name and author name.

Database

The database in example consists of a single table of three columns or fields. The database name is "books" and it contains information about **books names & authors**.

Table:books_details

ID	Book Name	Author
1.	Java I/O	Tim Ritchey
2.	Java & XML,2 Edition	Brett McLaughlin
3.	Java Swing, 2nd Edition	Dave Wood, Marc Loy

Start MYSQL prompt and type this SQL statement & press Enter-

MYSQL>CREATE DATABASE `books`;

This will create "books" database.

Now we create table a table "books_details" in database "books".

MYSQL>CREATE TABLE `books_details` (`id` INT(11) NOT NULL AUTO_INCREMENT , `book_name` VARCHAR(100) NOT NULL , `author` VARCHAR(100) NOT NULL , PRIMARY KEY (`id`)) TYPE = MYISAM ;

This will create a table "books_details" in database "books"

JSP Code

The following code contains html for user interface & the JSP back end-

```
<%@ page language="java" import="java.sql.*" %>
<%
        String driver = "org.gjt.mm.mysql.Driver";
        Class.forName(driver).newInstance();
        Connection con=null;
        ResultSetrst=null;
        Statement stmt=null;
        try  {
                String url="jdbc:mysql://localhost/books?user= user>&password=<password>";
                con=DriverManager.getConnection(url);
                stmt=con.createStatement();
        }
        catch(Exception e) {
                System.out.println(e.getMessage());
        }
        if(request.getParameter("action") != null){
                String bookname=request.getParameter("bookname");
                String author=request.getParameter("author");
                stmt.executeUpdate("insert into books_details(book_name,author)
               values('"+bookname+"','"+author+"')");
                rst=stmt.executeQuery("select * from books_details");
%>
                <html>
                <body>
                <center>
                        <h2> Books List </h2>
        <table border="1" cellspacing="0" cellpadding="0">
                        <tr>
                                <td> <b> S.No </b> </td>
                                <td> <b> Book Name </b> </td>
                                <td> <b> Author </b>  </td>
                        </tr>
        <%
                        int no=1;
                        while(rst.next()){
        %>

<tr>
```

```
                    <td> <%=no%> </td>
                    <td> <%=rst.getString("book_name")%> </td>
                <td> <%=rst.getString("author") </td>
            %>
                    </tr>
                    <%
                   no++;
        }
        rst.close();
        stmt.close();
        con.close();
        %>
        </table>
        </center>
        </body>
        </html>
<%}
else
  {  %>
        <html>
        <head>
                <title>Book Entry FormDocument</title>
                <script language="javascript">
                  function validate(objForm){
                        if(objForm.bookname.value.length==0){
                        alert("Please enter Book Name!");
                        objForm.bookname.focus();
                        return false;
                        }
                        if(objForm.author.value.length==0){
                        alert("Please enter Author name!");
                        objForm.author.focus();
                        return false;
                        }
                        return true;
                        }
                        </script>
                </head>
                <body>
```

```
                    <center>
<form action="BookEntryForm.jsp" method="post" name="entry" onSubmit="return validate(this)">
        <input type="hidden" value="list" name="action">
        <table border="1" cellpadding="0" cellspacing="0">
        <tr>
        <td>
        <table>
        <tr>
        <td colspan="2" align="center">
<h2> Book Entry Form </h2> </td>
        </tr>
        <tr>
        <td colspan="2"> </td>
        </tr>
        <tr>
        <td>Book Name:</td>
<td> <input name="bookname" type="text" size="50"> </td>
        </tr>
        <tr>
        <td> Author: </td> <td> <input name="author" type="text" size="50"> </td>
        </tr>
        <tr>
        <td colspan="2" align="center">
<input type="submit" value="Submit"> </td>
        </tr>
</table>
        </td>
</tr>
</table>
</form>
        </center>
        </body>
        </html>
<%}%>
```

Now we explain the above codes.

1. Declaring Variables:

Java is a strongly typed language which means, that variables must be explicitly declared before use and must be declared with the correct data types. In the above example code we declare some variables for making connection. These variables are-

Connection con=null;

ResultSetrst=null;

Statement stmt=null;

The objects of type Connection, ResultSet and Statement are associated with the Java sql. "con" is a Connection type object variable that will hold Connection type object. "rst" is a ResultSet type object variable that will hold a result set returned by a database query. "stmt" is a object variable of Statement .Statement Class methods allow to execute any query.

2. Connection to database:

The first task of this programmer is to load database driver. This is achieved using the single line of code:-

```
String driver = "org.gjt.mm.mysql.Driver";
Class.forName(driver).newInstance();
```

The next task is to make a connection. This is done using the single line of code :-

```
String url="jdbc:mysql://localhost/books?user=<userName>&password=<password>";
con=DriverManager.getConnection(url);
```

When url is passed into getConnection() method of DriverManager class it returns connection object.

3. Executing Query or Accessing data from database:

This is done using following code :-

```
stmt=con.createStatement(); //create a Statement object
rst=stmt.executeQuery("select * from books_details");
```

stmt is the Statement type variable name and **rst** is the RecordSet type variable. A query is always executed on a Statement object.

A Statement object is created by calling createStatement() method on connection object **con.**

The two most important methods of this Statement interface are executeQuery() and executeUpdate(). The executeQuery() method executes an SQL statement that returns a single ResultSet object. The executeUpdate() method executes an insert, update, and delete SQL statement. The method returns the number of records affected by the SQL statement execution.

After creating a Statement, a method executeQuery() or executeUpdate() is called on Statement object **stmt** and a SQL query string is passed in method executeQuery() or executeUpdate(). This will return a ResultSet**rst**related to the query string.

Reading values from a ResultSet:

```
while(rst.next()){
 %>
<tr><td><%=no%></td><td><%=rst.getString("book_name")%></td><td><%=rst.getString("a
uthor")%></td></tr>
```

```
<%
}
```

The ResultSet represents a table-like database result set. A ResultSet object maintains a cursor pointing to its current row of data. Initially, the cursor is positioned before the first row. Therefore, to access the first row in the ResultSet, you use the next() method. This method moves the cursor to the next record and returns true if the next row is valid, and false if there are no more records in the ResultSet object.

Other important methods are getXXX() methods, where XXX is the data type returned by the method at the specified index, including String, long, and int. The indexing used is 1-based. For example, to obtain the second column of type String, you use the following code:

```
resultSet.getString(2);
```

You can also use the getXXX() methods that accept a column name instead of a column index. For instance, the following code retrieves the value of the column LastName of type String.

```
resultSet.getString("book_name");
```

The above example shows how you can use the next() method as well as the getString() method. Here you retrieve the 'book_name' and 'author' columns from a table called 'books_details'. You then iterate through the returned ResultSet and print all the book name and author name in the format " book name | author " to the web page.